

ONEDRAFT

CAD-AI — Aria Powered

Identity, Access & Security System

Version 0.1

Status: Implementation Specification

Security Model: Identity + Organization + Project + Capability

Authentication: Bearer Token / OAuth 2.0 Compatible

Authorization: Role-Based + Scope-Based + Resource-Based

Audit: Immutable Security Events

Primary Boundary: Organization → Project → Resource

AI Layer: Aria Powered

1. SECURITY DESIGN PRINCIPLE

OneDraft shall treat security as an architectural subsystem rather than an application feature.

Every operation entering the platform must establish:

Who is making the request

Which organization the request belongs to

Which project is being accessed

Which resource is being accessed

Which capability is being requested

Whether the requested operation is authorized

Whether the operation must be audited

The security subsystem therefore becomes the authorization boundary between external actors and the OneDraft computational project model.

2. SECURITY ARCHITECTURE

The security architecture shall operate through the following chain:

CLIENT

↓
IDENTITY
↓
AUTHENTICATION
↓
SESSION / TOKEN
↓
ORGANIZATION MEMBERSHIP
↓
PROJECT ACCESS
↓
RESOURCE AUTHORIZATION
↓
CAPABILITY AUTHORIZATION
↓
DOMAIN SERVICE
↓
EVENT
↓
AUDIT

No external client may bypass this boundary to directly access project state.

3. SECURITY BOUNDARIES

OneDraft shall establish the following security boundaries:

INTERNET
↓
EDGE / API GATEWAY
↓
AUTHENTICATION
↓
APPLICATION API
↓
DOMAIN SERVICES
↓
WORKERS
↓
DATABASE / OBJECT STORAGE

Internal services shall authenticate to one another.

A network location alone shall not constitute authorization.

4. SECURITY PRINCIPLES

The implementation shall follow:

Least privilege

Explicit authorization

Deny by default

Resource isolation

Tenant isolation

Credential minimization

Immutable auditing

Short-lived credentials

Separation of duties

Defense in depth

Re-authentication for sensitive operations

5. IDENTITY DOMAIN

The identity subsystem shall manage:

Users
Organizations
Memberships
Roles
Credentials
Sessions
Service identities
API clients
Security events

The existing identity schema remains the authoritative source for application-level identity.

6. USER IDENTITY

The existing:

`identity.users`

table remains the primary application identity record.

The user identity shall not contain authentication secrets directly unless explicitly required by the selected identity implementation.

Authentication credentials should remain separated from project-domain records.

7. USER STATUS

Initial user statuses:

INVITED
ACTIVE
SUSPENDED
DISABLED
DELETED

Only:

ACTIVE

users may authenticate normally.

Suspended or disabled identities must fail authorization even where a previously issued credential has not yet expired.

8. ORGANIZATION SECURITY BOUNDARY

The organization is the primary tenant boundary.

Organization
↓
Members
↓
Projects
↓
Project Resources

A user must never obtain access to another organization's project merely because the resource identifier is known.

9. PROJECT SECURITY BOUNDARY

Every project request shall establish:

organization_id
project_id
user_id

The API shall verify that:

user
↓
member of organization
↓
authorized for project

before executing the requested domain operation.

10. MEMBERSHIP ROLES

Initial organization roles:

OWNER
ADMIN
MEMBER
VIEWER
BILLING

Project-specific roles may additionally include:

PROJECT_ADMIN
PROJECT_EDITOR
PROJECT_REVIEWER
PROJECT_VIEWER

Roles are permissions containers and shall not replace explicit capability checks.

11. CAPABILITY MODEL

The platform shall use capability scopes such as:

project:read
project:write

model:read
model:write

aria:execute
aria:approve

geometry:read
geometry:generate

drawing:read
drawing:generate

redline:read
redline:write

compliance:read
compliance:execute

validation:read
validation:execute

document:read
document:generate

acceptance:write

event:read

admin:organization

admin:project

admin:security

12. AUTHORIZATION MODEL

Authorization shall evaluate three dimensions:

IDENTITY

+

RESOURCE

+

CAPABILITY

Conceptually:

Can User X
perform Capability Y
against Resource Z?

A valid authentication token alone is insufficient.

13. AUTHORIZATION DECISION

Authorization decisions shall resolve to:

ALLOW

DENY

REAUTH_REQUIRED

The application shall not infer authorization from client-side state.

The server remains authoritative.

14. AUTHENTICATION

The MVP shall use bearer-token authentication compatible with:

Authorization: Bearer <token>

The authentication layer shall validate:

issuer
audience
signature
expiration
issued-at
subject
scopes
token status

15. TOKEN PRINCIPLE

Tokens shall contain the minimum information required to establish identity and authorization context.

Sensitive project information shall not be embedded into bearer tokens unnecessarily.

Project authorization remains a server-side decision.

16. TOKEN EXPIRATION

Access tokens shall be short-lived.

Long-lived credentials shall not be used as ordinary access tokens.

Refresh mechanisms, where implemented, shall use protected refresh credentials and revocation controls.

17. SESSION MANAGEMENT

Authenticated sessions shall support:

session_id
user_id
issued_at
expires_at
last_activity
client_metadata
status
revoked_at

Session statuses:

ACTIVE
EXPIRED
REVOKED

18. SESSION REVOCATION

Security administrators must be able to revoke active sessions.

Revocation events shall be recorded.

A revoked session must no longer authorize API access.

19. MULTI-FACTOR AUTHENTICATION

The architecture shall support MFA without requiring the domain model to change.

Sensitive operations may require elevated authentication.

Potential sensitive operations include:

- organization administration
- security administration
- credential changes
- service-account changes
- billing administration
- acceptance authorization
- production configuration

20. REAUTHENTICATION

Operations designated as high assurance may require:

- recent authentication
- +
- required capability

This prevents a long-lived authenticated browser session from automatically authorizing high-impact operations.

21. SERVICE IDENTITIES

Internal workers and services shall not authenticate as ordinary users.

The system shall support service identities:

- service_id
- name
- status
- scopes
- created_at

updated_at

Examples:

geometry-worker
validation-worker
documentation-worker
job-worker
event-consumer
artifact-service

22. SERVICE-TO-SERVICE AUTHENTICATION

Internal services shall authenticate through controlled service credentials.

The architecture becomes:

```
SERVICE A
  ↓
SERVICE IDENTITY
  ↓
AUTHORIZED CAPABILITY
  ↓
SERVICE B
```

A worker shall receive only the permissions necessary to perform its assigned function.

23. WORKER SECURITY

Workers shall not receive unrestricted database credentials.

The preferred architecture is:

```
JOB
  ↓
WORKER
  ↓
DOMAIN SERVICE
  ↓
AUTHORIZED OPERATION
```

Workers may consume jobs while domain services retain control over state mutation.

24. ARIA SECURITY BOUNDARY

Aria shall operate entirely inside the OneDraft capability model.

Aria shall not receive unrestricted credentials to:

- PostgreSQL
- Object Storage
- Identity Provider
- Billing
- Production Infrastructure

Aria requests capabilities.

The application decides whether those capabilities may execute.

25. ARIA AUTHORIZATION

An Aria command shall establish:

- user_id
- organization_id
- project_id
- revision_id
- command_id
- requested_capability

The command boundary shall then evaluate:

- identity authorization
- +
- project authorization
- +
- command authorization
- +
- model constraints
- +
- revision state

26. ARIA APPROVAL BOUNDARY

Operations may be divided into:

- READ
- PROPOSE
- EXECUTE
- APPROVE
- PUBLISH

This allows OneDraft to distinguish between:

Aria proposing a change

and:

Aria executing an authorized change

27. PRIVILEGED OPERATIONS

Privileged operations shall be explicitly enumerated.

Examples:

DELETE_PROJECT
CHANGE_ORGANIZATION_OWNER
CHANGE_SECURITY_POLICY
CREATE_SERVICE_IDENTITY
ROTATE_SERVICE_CREDENTIAL
MODIFY_PRODUCTION_CONFIGURATION
PUBLISH_FINAL_PACKAGE
RECORD_ACCEPTANCE

No privileged operation shall be activated merely because a user is authenticated.

28. RESOURCE AUTHORIZATION

Authorization shall be applied recursively.

For example:

Project
↓
Building
↓
Level
↓
Space
↓
Element
↓
Geometry

Access to an element derives from access to the containing project unless an explicit resource policy states otherwise.

29. ARTIFACT AUTHORIZATION

Stored artifacts shall inherit project authorization.

An artifact reference shall therefore resolve through:

```
artifact
↓
project
↓
organization
↓
authorized principal
```

Direct knowledge of an object-storage key shall not grant access.

30. DOCUMENT AUTHORIZATION

Generated document packages shall be protected by the same project boundary.

A document package shall identify:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
```

Access requires authorization to the associated project.

31. EVENT AUTHORIZATION

Events may contain sensitive project information.

Event consumers shall therefore receive only events appropriate to their identity and scope.

External clients shall not automatically receive internal event-bus traffic.

32. EVENT SECURITY

Security-sensitive events shall include:

```
actor_id
organization_id
project_id
event_type
```

timestamp
request_id
correlation_id
source

Where appropriate:

ip_hash
client_id
service_id

may also be recorded.

33. SECURITY EVENTS

The security subsystem shall generate immutable security events for:

LOGIN_SUCCESS
LOGIN_FAILURE
LOGOUT
SESSION_REVOKED
MFA_ENABLED
MFA_DISABLED
PASSWORD_CHANGED
CREDENTIAL_CREATED
CREDENTIAL_REVOKED
ROLE_CHANGED
MEMBERSHIP_CHANGED
AUTHORIZATION_DENIED
PRIVILEGED_OPERATION
SERVICE_AUTHENTICATION
SECURITY_POLICY_CHANGED

34. AUDIT INTEGRATION

Security events shall flow through the established event architecture.

SECURITY ACTION
↓
SECURITY EVENT
↓
EVENT BUS
↓
AUDIT

The existing append-only audit architecture remains authoritative for historical security records.

35. REQUEST IDENTITY

Every API request shall receive:

`request_id`

The request identifier shall propagate through:

```
API
↓
DOMAIN SERVICE
↓
JOB
↓
WORKER
↓
EVENT
↓
AUDIT
```

This establishes end-to-end traceability.

36. CORRELATION ID

Long-running operations shall additionally use:

`correlation_id`

For example:

```
ARIA COMMAND
↓
JOB
↓
GEOMETRY
↓
VALIDATION
↓
DRAWING
↓
DOCUMENT
```

All associated operations can therefore be reconstructed as one execution chain.

37. IDEMPOTENCY SECURITY

Mutating requests shall continue to support:

Idempotency-Key

The security subsystem shall bind idempotency records to the authenticated principal and appropriate resource context.

This prevents one identity from replaying another identity's request context.

38. API GATEWAY SECURITY

The API gateway shall perform preliminary:

- TLS termination
- token validation
- request size limits
- rate limiting
- request identification
- basic threat filtering

Domain authorization remains inside application services.

39. RATE LIMITING

Rate limits shall be applied by appropriate dimensions:

- IP
- user
- organization
- service identity
- API endpoint
- operation type

High-impact operations may have lower limits than read operations.

40. ARIA RATE LIMITING

Aria operations shall have explicit execution controls.

Limits may apply to:

- commands per minute
- concurrent commands
- geometry jobs
- document generation jobs
- validation jobs

The objective is to prevent uncontrolled automated mutation.

41. INPUT VALIDATION

All external inputs shall be validated before reaching domain services.

Validation includes:

- type
- format
- length
- range
- enumeration
- resource ownership
- authorization
- schema

Client-side validation is supplementary only.

42. COMMAND VALIDATION

An incoming command shall pass through:

- authentication
 - ↓
- authorization
 - ↓
- schema validation
 - ↓
- project state validation
 - ↓
- constraint validation
 - ↓
- domain execution

A malformed or unauthorized command shall never reach the mutation boundary.

43. SECRET MANAGEMENT

Secrets shall not be stored in:

- source code
- Git repositories
- OpenAPI documents
- database seed data
- frontend bundles
- logs
- event payloads

Secrets shall be supplied through controlled secret-management infrastructure.

44. ENVIRONMENT SEPARATION

OneDraft shall maintain separate environments:

DEVELOPMENT
TEST
STAGING
PRODUCTION

Credentials and signing keys must not be shared between environments.

45. PRODUCTION ACCESS

Production administrative access shall be restricted.

Production access should require:

authenticated identity
+
explicit role
+
appropriate capability
+
audit event

46. DATABASE SECURITY

Application services shall use restricted database roles.

Conceptually:

READ SERVICE
WRITE SERVICE
MIGRATION SERVICE
ADMIN SERVICE

Database credentials shall not be embedded in frontend applications.

47. ROW-LEVEL ISOLATION

Where appropriate, PostgreSQL Row-Level Security may reinforce organization/project isolation.

The primary authorization decision remains in the application layer.

Defense-in-depth may therefore become:

48. OBJECT STORAGE SECURITY

Object storage shall use private buckets/containers by default.

Artifacts shall be retrieved through authorized application mechanisms.

Temporary signed URLs may be generated for controlled downloads.

Signed URLs shall have limited lifetimes.

49. ENCRYPTION

Data shall be protected:

in transit

through modern TLS.

Sensitive stored information shall use appropriate:

encryption at rest

provided by the selected infrastructure.

50. PASSWORD SECURITY

Where OneDraft directly manages passwords, passwords shall never be stored in plaintext.

They shall be processed through a modern password hashing mechanism.

However, the preferred architecture is to delegate authentication to a dedicated identity provider where practical.

51. API CLIENT CREDENTIALS

External integrations may receive API credentials scoped to:

organization

project
capability

Credentials shall support:

creation
rotation
expiration
revocation
audit

52. WEBHOOK SECURITY

Outbound webhooks shall use signed payloads.

Each webhook shall provide:

event_id
timestamp
signature
payload

Consumers shall verify the signature before accepting the event.

53. EVENT REPLAY PROTECTION

Event consumers shall protect against duplicate and replayed events.

Each event shall have a unique:

event_id

Consumers shall maintain sufficient processing state to safely recognize previously processed events.

54. SECURITY POLICY

The system shall support organization-level security policy configuration.

Initial policy categories:

session_policy
authentication_policy
mfa_policy
api_policy
artifact_policy
retention_policy
integration_policy

Policy changes shall be audited.

55. SECURITY POLICY HIERARCHY

Policy resolution shall follow:

```
PLATFORM POLICY
↓
ORGANIZATION POLICY
↓
PROJECT POLICY
↓
RESOURCE POLICY
```

A lower-level policy may not weaken a mandatory platform security requirement.

56. DATA ACCESS LOGGING

Sensitive reads may require audit logging in addition to mutating operations.

Examples:

```
document download
project export
security configuration access
customer acceptance record access
professional review access
```

57. EXPORT SECURITY

Project exports shall require authorization.

Exports shall preserve:

```
project identity
revision identity
model version
document provenance
```

The export operation shall generate an audit event.

58. DELETION SECURITY

Deletion shall be treated as a privileged state transition.

For project-level objects:

ACTIVE
↓
DELETED

Normal application operation shall prefer soft deletion.

Historical records remain protected from ordinary deletion.

59. RETENTION

Retention policies shall distinguish:

active project state
historical revisions
audit events
security events
generated documents
temporary artifacts
job records

The platform shall not use one universal deletion policy for all classes of data.

60. SECURITY INCIDENT STATE

The architecture shall support security incident markers without altering project history.

An incident may reference:

organization
project
user
service
request
event
artifact
job
command

This permits security investigations without corrupting the underlying project model.

61. COMPROMISED CREDENTIAL RESPONSE

Credentials shall be capable of immediate revocation.

The response sequence becomes:

```
DETECT
↓
REVOKE
↓
INVALIDATE SESSIONS
↓
ROTATE CREDENTIALS
↓
AUDIT
↓
REVIEW
```

The project model itself remains unchanged unless a separately authorized domain operation occurs.

62. SECURITY AND REVISION INTEGRITY

Security operations must not silently create architectural model revisions.

A security event and a model revision are distinct concepts.

SECURITY EVENT $\neq$ MODEL REVISION

This preserves the separation between:

what happened to the system

and:

what changed in the architectural project model

63. SECURITY AND COMPLIANCE

The security subsystem shall integrate with the existing compliance and validation architecture without replacing it.

For example:

```
SECURITY POLICY
↓
AUTHORIZED OPERATION
↓
```

DOMAIN COMMAND



COMPLIANCE VALIDATION



MODEL REVISION

Security authorization occurs before domain execution.

Regulatory validation evaluates the resulting project state.

64. SECURITY AND ACCEPTANCE

Customer acceptance remains a domain operation.

The security subsystem determines whether the authenticated principal has the capability to submit acceptance.

The acceptance record remains part of the established revision/document provenance chain.

65. SECURITY API GROUP

The initial API surface shall include:

Authentication
Sessions
Users
Organizations
Memberships
Roles
Capabilities
Service Identities
API Credentials
Security Events
Security Policies

66. SECURITY ENDPOINTS

Conceptual endpoints include:

GET /api/v1/me

GET /api/v1/organizations

GET /api/v1/organizations/{organization_id}

GET /api/v1/organizations/{organization_id}/members

POST /api/v1/organizations/{organization_id}/members

```
GET    /api/v1/projects/{project_id}/security
GET    /api/v1/projects/{project_id}/security/events
GET    /api/v1/security/sessions
DELETE /api/v1/security/sessions/{session_id}

GET    /api/v1/security/policies
PUT    /api/v1/security/policies

GET    /api/v1/service-identities
POST   /api/v1/service-identities

POST   /api/v1/api-credentials
DELETE /api/v1/api-credentials/{credential_id}
```

Exact endpoint implementation shall be established in the OpenAPI contract.

67. SECURITY RESPONSE

Successful security operations shall use the existing OneDraft response envelope:

```
{
  "data": {},
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```

Security-sensitive responses shall avoid returning unnecessary credential material.

68. AUTHORIZATION ERROR

Unauthorized requests shall return a standardized error.

Example:

```
{
  "error": {
    "code": "AUTHORIZATION_DENIED",
    "message": "The authenticated principal is not authorized for this operation.",
    "resolution": "Request the required project capability."
  },
  "metadata": {
    "request_id": "uuid",
    "timestamp": "ISO-8601"
  }
}
```


The response must not disclose sensitive authorization internals.

69. SECURITY DATA MODEL

The security subsystem may introduce:

```
identity.sessions  
identity.roles  
identity.role_permissions  
identity.service_identities  
identity.api_credentials  
identity.security_events  
identity.security_policies
```

Existing:

```
identity.users  
identity.organizations  
identity.organization_members
```

remain foundational.

70. SECURITY EVENT MODEL

Conceptually:

```
identity.security_events
```

Fields:

```
id UUID PRIMARY KEY  
event_type VARCHAR(128) NOT NULL  
user_id UUID  
organization_id UUID  
project_id UUID  
service_identity_id UUID  
session_id UUID  
request_id UUID  
correlation_id UUID  
source VARCHAR(64)  
payload JSONB NOT NULL  
timestamp TIMESTAMPTZ NOT NULL
```

Security events are append-only.

71. SERVICE IDENTITY MODEL

Conceptually:

`identity.service_identities`

Fields:

```
id UUID PRIMARY KEY
name VARCHAR(255) NOT NULL
description TEXT
status VARCHAR(32) NOT NULL
scopes JSONB NOT NULL
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

72. API CREDENTIAL MODEL

Conceptually:

`identity.api_credentials`

Fields:

```
id UUID PRIMARY KEY
organization_id UUID
service_identity_id UUID
name VARCHAR(255) NOT NULL
credential_hash VARCHAR(255) NOT NULL
scopes JSONB NOT NULL
expires_at TIMESTAMPTZ
last_used_at TIMESTAMPTZ
status VARCHAR(32) NOT NULL
created_at TIMESTAMPTZ NOT NULL
revoked_at TIMESTAMPTZ
```

Raw credential values shall not be stored after issuance.

73. ROLE-PERMISSION MODEL

Conceptually:

`identity.roles`
`identity.role_permissions`

A role maps to capabilities:

OWNER
↓

```
organization:admin
project:read
project:write
...
```

The actual permission matrix shall be maintained as executable configuration rather than duplicated across controllers.

74. SECURITY MIDDLEWARE

Every protected API request shall pass through security middleware:

```
REQUEST
↓
REQUEST ID
↓
TOKEN VALIDATION
↓
IDENTITY RESOLUTION
↓
ORGANIZATION RESOLUTION
↓
PROJECT RESOLUTION
↓
AUTHORIZATION
↓
DOMAIN SERVICE
```

75. SECURITY CONTEXT

The domain layer shall receive an immutable security context.

Conceptually:

```
SecurityContext(
    user_id,
    organization_id,
    project_id,
    scopes,
    roles,
    request_id,
    correlation_id,
)
```

Domain services shall not reconstruct authorization from arbitrary request payloads.

76. DOMAIN SERVICE RULE

Domain services shall receive:

validated command
+
security context

rather than raw HTTP requests.

This maintains the separation:

HTTP
↓
API
↓
Security
↓
Domain

77. FRONTEND SECURITY

The frontend shall never be treated as a trusted authorization layer.

The frontend may:

hide unavailable controls
display permitted capabilities
manage user interaction

but the backend must independently enforce authorization.

78. SECURITY TESTING

The initial automated security test suite shall include:

unauthenticated request
expired token
revoked token
invalid token
wrong organization
wrong project
insufficient role
insufficient scope
cross-project access
cross-organization access
revoked session
replayed request
invalid service credential
expired service credential

unauthorized Aria command
unauthorized document download
unauthorized acceptance

79. SECURITY INTEGRATION TEST

The first complete security integration test shall execute:

1. Create Organization
2. Create User A
3. Create User B
4. Create Project
5. Add User A as OWNER
6. Add User B as VIEWER
7. Authenticate User A
8. Authenticate User B
9. Verify User A can modify the project
10. Verify User B cannot modify the project
11. Verify User B can read authorized project state
12. Attempt cross-organization access
13. Verify authorization denial
14. Create service identity
15. Issue scoped credential
16. Execute authorized worker operation
17. Attempt unauthorized worker operation
18. Verify denial
19. Submit Aria command
20. Verify Aria capability authorization
21. Record security event
22. Revoke credential
23. Verify subsequent request fails
24. Retrieve security audit history

This becomes the foundational security regression test.

80. SECURITY OBSERVABILITY

Security telemetry shall integrate with the established runtime and event architecture.

Important metrics include:

- authentication failures
- authorization denials
- active sessions
- revoked sessions
- credential issuance
- credential revocation
- privileged operations
- Aria authorization failures
- service authentication failures
- cross-tenant access attempts

81. SECURITY HEALTH

The runtime health system shall expose security subsystem health without exposing secrets.

Conceptually:

- identity_provider
- token_validation
- session_store
- authorization_service
- secret_store
- audit_pipeline

Each may report:

- HEALTHY
- DEGRADED
- FAILED

82. FAILURE PRINCIPLE

Security failures shall fail closed.

If authorization cannot be established:

- DENY

The system shall not assume permission because an authorization dependency is unavailable.

83. EVENT BUS INTEGRATION

The security subsystem shall publish events through the established event bus.

Examples:

```
security.user.authenticated
security.authorization.denied
security.session.revoked
security.credential.created
security.credential.revoked
security.role.changed
security.policy.changed
```

Consumers may include:

```
audit
observability
notification
security monitoring
administration
```

84. JOB SYSTEM INTEGRATION

Jobs shall carry security provenance:

```
job_id
requested_by
organization_id
project_id
request_id
correlation_id
required_scopes
```

A worker shall verify that the job remains valid before performing a sensitive operation.

85. ARTIFACT SYSTEM INTEGRATION

Artifacts shall remain private by default.

Artifact access shall resolve through:

```
authenticated principal
↓
project authorization
```

↓
artifact authorization
↓
temporary access

86. RUNTIME ORCHESTRATOR INTEGRATION

The Runtime Orchestrator shall be responsible for preserving security context across asynchronous execution.

API REQUEST
↓
ORCHESTRATOR
↓
JOB
↓
WORKER
↓
DOMAIN SERVICE

The security context must remain attributable to the initiating principal.

87. SECURITY PROVENANCE CHAIN

OneDraft shall therefore establish:

USER
↓
SESSION
↓
REQUEST
↓
COMMAND
↓
JOB
↓
WORKER
↓
MODEL MUTATION
↓
REVISION
↓
EVENT
↓
AUDIT

This provides an end-to-end provenance chain for authorized computational activity.

88. NO DIRECT TRUST ESCALATION

No component may convert:

read

into:

write

or:

propose

into:

execute

without an explicit authorization transition.

Likewise:

project access

does not automatically confer:

organization administration

89. SECURITY AND SYSTEM OF RECORD

The established OneDraft system-of-record hierarchy remains unchanged.

Security information records **access and authority events**.

The Project Model remains authoritative for architectural state.

Therefore:

SECURITY RECORD

≠

PROJECT MODEL

Both may be linked through identifiers and audit provenance.

90. SECURITY RECONSTRUCTION TEST

Given:

user_id
organization_id
project_id
request_id
correlation_id

the platform should eventually reconstruct:

who initiated the operation
what capability was requested
whether authorization succeeded
which command executed
which job processed it
which model revision resulted
which events were generated

This establishes security provenance across the complete OneDraft execution lifecycle.

91. MVP SECURITY SUCCESS CRITERIA

The security implementation is successful when it can:

Authenticate users

Authorize organization membership

Authorize project access

Enforce capability scopes

Enforce role permissions

Authenticate services

Protect worker operations

Protect artifacts

Protect documents

Authorize Aria commands

Revoke sessions

Revoke credentials

Record security events

Propagate request identity

Propagate correlation identity

Prevent cross-organization access

Prevent unauthorized project mutation

Fail closed when authorization cannot be established

92. IMPLEMENTATION ARTIFACTS

The next implementation package shall consist of:

Artifact A

`security_models.py`

Identity, roles, scopes, sessions, credentials and security-context models.

Artifact B

`authorization_service.py`

Central authorization and capability evaluation.

Artifact C

`security_middleware.py`

Authentication, request identity and security-context construction.

Artifact D

`security_events.py`

Security event definitions and publication.

Artifact E

`identity_migration.sql`

Additional persistence required by the security subsystem.

Artifact F

`security_integration_tests.py`

The first executable security regression suite.

93. IMPLEMENTATION ORDER

The implementation sequence shall be:

```
IDENTITY MODELS
↓
ROLE / SCOPE MODEL
↓
AUTHENTICATION ADAPTER
↓
SECURITY CONTEXT
↓
AUTHORIZATION SERVICE
↓
API MIDDLEWARE
↓
SERVICE IDENTITIES
↓
CREDENTIAL MANAGEMENT
↓
SECURITY EVENTS
↓
EVENT BUS INTEGRATION
↓
RUNTIME INTEGRATION
↓
JOB INTEGRATION
↓
ARTIFACT INTEGRATION
↓
SECURITY TESTS
```

94. ARCHITECTURAL INTEGRATION

The OneDraft stack now becomes:

```
PROJECT DOMAIN
↓
GEOMETRY & PARAMETRIC MODEL
↓
COMPLIANCE & REGULATORY ENGINE
↓
VALIDATION ENGINE
↓
REVISION & REDLINE SYSTEM
↓
DOCUMENTATION & DRAWING ENGINE
↓
RUNTIME ORCHESTRATOR
↓
JOB & WORKER EXECUTION
↓
ARTIFACT STORAGE
↓
```

INTEGRATION & EVENT BUS



IDENTITY, ACCESS & SECURITY

The security layer surrounds these subsystems rather than becoming another isolated domain component.

95. SECURITY BOUNDARY OF ARIA

Aria remains the intelligence layer.

Aria does not become the security authority.

The architecture therefore remains:

USER



IDENTITY



AUTHORIZATION



ARIA



STRUCTURED COMMAND



DOMAIN SERVICE



PROJECT MODEL

This prevents the intelligence layer from becoming an uncontrolled privileged actor.

96. FINAL ARCHITECTURAL DECISION

OneDraft shall not rely upon:

frontend authorization

+

hidden URLs

+

database secrecy

+

AI discretion

+

network location

as its security model.

OneDraft shall rely upon:

IDENTITY

- +
AUTHENTICATION
- +
ORGANIZATION ISOLATION
- +
PROJECT AUTHORIZATION
- +
CAPABILITY CONTROL
- +
SERVICE IDENTITY
- +
AUDIT
- +
EVENT PROVENANCE

Security therefore becomes an enforceable property of the OneDraft execution architecture.

97. VERSION 0.1 ENGINEERING STATE

The OneDraft architecture now includes:

Product definition

System architecture

Technology architecture

Domain model

Persistence model

OpenAPI contract

Geometry engine

Parametric model

Compliance engine

Validation engine

Revision system

Redline system

Documentation engine

Drawing generation

Runtime orchestration

Job execution

Worker execution

Artifact storage

Integration layer

Event bus

Identity

Authentication

Authorization

Service security

Security provenance

Audit integration

The architecture therefore now has an explicit trust boundary surrounding the computational project model.

98. NEXT IMPLEMENTATION ARTIFACTS

The immediate executable foundation shall be:

`security_models.py`

`authorization_service.py`

`security_middleware.py`

`security_events.py`

`identity_migration.sql`

`security_integration_tests.py`

These artifacts shall connect the security boundary to the already-established:

API

+

Domain Services

+

Runtime Orchestrator

+

Job System

+

Worker System

+

Artifact Storage

+

Event Bus

99. NEXT STACK

The next OneDraft subsystem may therefore proceed directly into the **Observability, Telemetry & System Health subsystem** without redefining the underlying:

- project
- geometry
- compliance
- validation
- revision
- documentation
- runtime orchestration
- job execution
- artifact storage
- event bus
- identity
- access
- security

architecture.

The next layer will establish the machine-visible operational state of the entire OneDraft platform, including:

- metrics
- logs
- traces
- health checks
- job telemetry
- worker telemetry
- API telemetry
- event-bus telemetry
- security telemetry
- AI execution telemetry
- geometry telemetry
- validation telemetry
- document-generation telemetry
- system alerts